

Designing Applications with AI Agents

Patterns, Prompts and Architectural Decisions
for the era of autonomy.



Architecture

Structural decisions and trade-offs.



Prompting

Context engineering and guardrails.



Orchestration

Tools, memory and agentic patterns.

Questions you're probably asking yourself

Q.01 How do I design good tools for my agent?

Q.02 What kind of memory do I need?

Q.03 Should I use RAG?

Q.04 Single-agent or multi-agent?

Q.05 What architecture should I use?

Q.06 Do I need to fine-tune?

Q.07 How do I get the agent to learn over time?

Q.08 Do I really need an agent... or is a script enough?

All of these are architecture questions. Today we'll learn to answer them.

What we'll cover today

// 01

Software Architecture

Definition, the trade-off law and the architect's role.

// 02

Project Components

Requirements, constraints and quality attributes.

// 03

Agentic Systems

Models, prompts, tools, memory, patterns and trade-offs.

// 04

Impact on Development

How day-to-day changes and why the architect matters more.

Software Architecture

Structural decisions that sustain the product.

What is software architecture?

OPERATIONAL DEFINITION

The set of structural decisions —expensive to change— that define how a system meets its functional requirements, constraints and quality attributes.

Architecture is not a diagram: it is the reasoning behind the decisions and the trade-offs assumed.



Structure

Components, boundaries, interfaces and dependencies.



Reasoning

Why we picked X over Y: criteria, context and documented trade-offs.

“ **The first rule of software architecture
is that everything in software architecture
is a trade-off.**

— *Mark Richards & Neal Ford*

There is no silver bullet

Every decision leaves something behind.

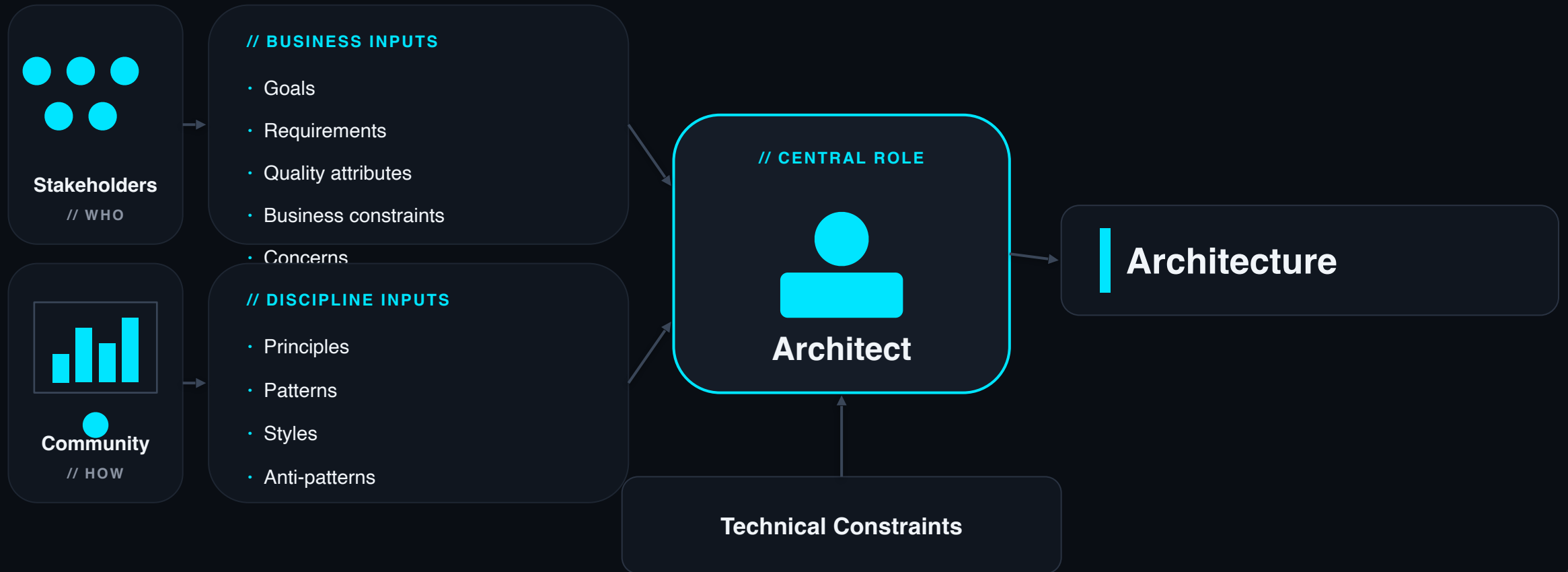
It depends on context

What's optimal today may be debt tomorrow.

Make trade-offs explicit

Document what you gain and what you cede.

What does a software architect do?



Project Components

The inputs that shape the architecture.

The 4 inputs of every architecture

// 01

Functional Requirements

What the system must do.

// 02

Business Constraints

Rules, budget, deadlines, compliance.

// 03

Technical Constraints

Stack, integrations, infra, organization.

// 04

Quality Attributes

The -ilities that govern the system.

Functional Requirements

What the system must do.

- Features, flows and use cases.
- Observable behavior (user stories, specs, BDD).
- The “what”. Which problem are we solving?
- On their own they don't define architecture, but they set the scope.

EXAMPLE

- Register users
- Process payments
- Generate PDF reports
- Notify via email
- Search products by text

Business Constraints

What the business imposes before choosing technology.

- Budget and cost model.
- Time-to-market and contractual deadlines.
- Regulation and compliance (GDPR, HIPAA, PCI...).
- Internal policies, licenses and vendor lock-in.
- Target market and geographies.

IMPACT

The business tells you where you CAN'T go. It defines the space of possible solutions and rules out options that are technically valid but unviable.

Technical Constraints

The technological ecosystem the system lives in.

- Existing stack and technical debt.
- Mandatory integrations (ERP, CRM, legacy, APIs).
- Infrastructure and cloud provider.
- Team knowledge and size (Conway's Law).
- Third-party SLAs and operational limits.

TIP

The real cost of a technical decision includes operating it, observing it, evolving it and hiring people capable of maintaining it for years.

Quality Attributes

The -ilities that govern how the system behaves.

■ Performance

■ Scalability

■ Reliability

■ Availability

■ Security

■ Observability

■ Maintainability

■ Testability

■ Deployability

■ Portability

■ Usability

■ Cost-efficiency

They conflict with each other. Prioritizing them is the most important architectural decision.

What decides the architecture?



KEY

Quality attributes and constraints are what most affect the architecture.

Functionality can be implemented many ways; the -ilities are what rule options out.

Agentic Systems

When the system reasons, decides and acts.

Agent vs Non-Agent

// AGENT

Autonomous system that reasons and acts

- Analyzes data autonomously.
- Interprets its environment.
- Decides based on context.
- Plans multi-step actions.
- Learns from feedback.

// NOT AN AGENT

Script or workflow in disguise

- Executes deterministic steps.
- Fixed flow with no reasoning.
- No adaptation to novelty.
- Rule-based chatbots / decision trees.
- Traditional RPA (UiPath, Zapier).

Many systems labeled “agents” are, in reality, workflows with marketing.

What makes agents possible today?

FRONTIER LLMs

GPT-5 · Claude Opus 4.1 · Llama 3 · Gemini Ultra · DeepSeek V3

Models capable of reasoning, using tools and planning multi-step actions.

NLU

Understands ambiguous natural language.

Context

Coherence in long interactions.

Tool use

Invokes functions to act.

Planning

Decomposes goals into multi-step actions.

Unstructured

Emails, docs, logs, reports.

Multimodal

Text, image, audio, video.

LLMs went from “completing text” to “making executable decisions”.

From handcrafted ML pipelines to pre-trained agents

What once demanded months of ML engineering now comes built into the model.

// BEFORE · EVERY AI WAS A PROJECT

Handcrafted ML pipeline

- 01 Collect datasets
- 02 Clean and label
- 03 Feature engineering
- 04 Train and tune
- 05 Validate and retrain
- 06 Deploy + monitor drift

Weeks to months · Specialists · Expensive and fragile

// TODAY · FRONTIER LLMs

Pre-trained models

GPT-5

Claude Opus 4.1

Llama 3

Gemini Ultra

DeepSeek V3

BUILT-IN

- Long context and language
- Tool use (function calling)
- Unstructured data
- Decisions under ambiguity
- Multi-step planning
- Multimodal: text, image, audio

From ML project to API integration

Types of agents you'll see in the wild

// 01

Business-task

Next-gen RPA

UiPath, Power Automate,
Zapier

// 02

Conversational

Chatbots and support

Virtual assistants,
customer support

// 03

Research

Search and synthesis

Perplexity, Elicit

// 04

Analytics

Insights on top of data

Power BI Copilot,
Glean

// 05

Developer

Coding assistants

Cursor, Claude Code,
Windsurf, Copilot

// 06

Domain-specific

Vertical experts

Harvey (legal),
Hippocratic AI (healthcare)

// 07

Browser-using

Browse the web for you

Automate
web interfaces

// 08

Voice / Video

Multimodal

Phone support,
video summarization

Do you really need an agent?

Before building an agent, evaluate 4 factors:

Input variability

Predictable or
open-ended?

Reasoning complexity

Linear or multi-step
with ambiguity?

Performance / compliance

Latency critical?
Auditability?

Continuous maintenance

Can you operate
and evaluate?

USE // 01

Traditional code

Predictable inputs and enumerable
outputs.

USE // 02

Deterministic workflow

Finite branches and clear rules.

USE // 03

Chatbot / RAG

Q&A over content, no external
decisions.

USE // 04

Autonomous agent

Novel inputs + multi-step + learning.

An agent is only justified when everything else fails.

Traditional code · Workflow · Agent

Trait	Traditional code	Workflow	Autonomous agent
Input structure	Fully predictable	Mostly predictable, finite branches	Highly unstructured / novel
Explainability	Full transparency	Auditable branch by branch	Black box: needs extra tooling
Latency	Ultra low	Moderate	Higher (LLM inference)
Adaptability	None	Limited	High (learns from feedback)
An agent is the most expensive option. Make sure the problem actually requires one.			

Quality attributes for agentic systems



Scalability

Distribution, autoscaling and parallelizable algorithms. Ex.: an agent processing 10 tickets/min must survive spikes of 1,000 without collapsing.



Modularity

Tools, prompts and memory as replaceable modules. You evolve parts without rewriting the agent.



Continuous learning

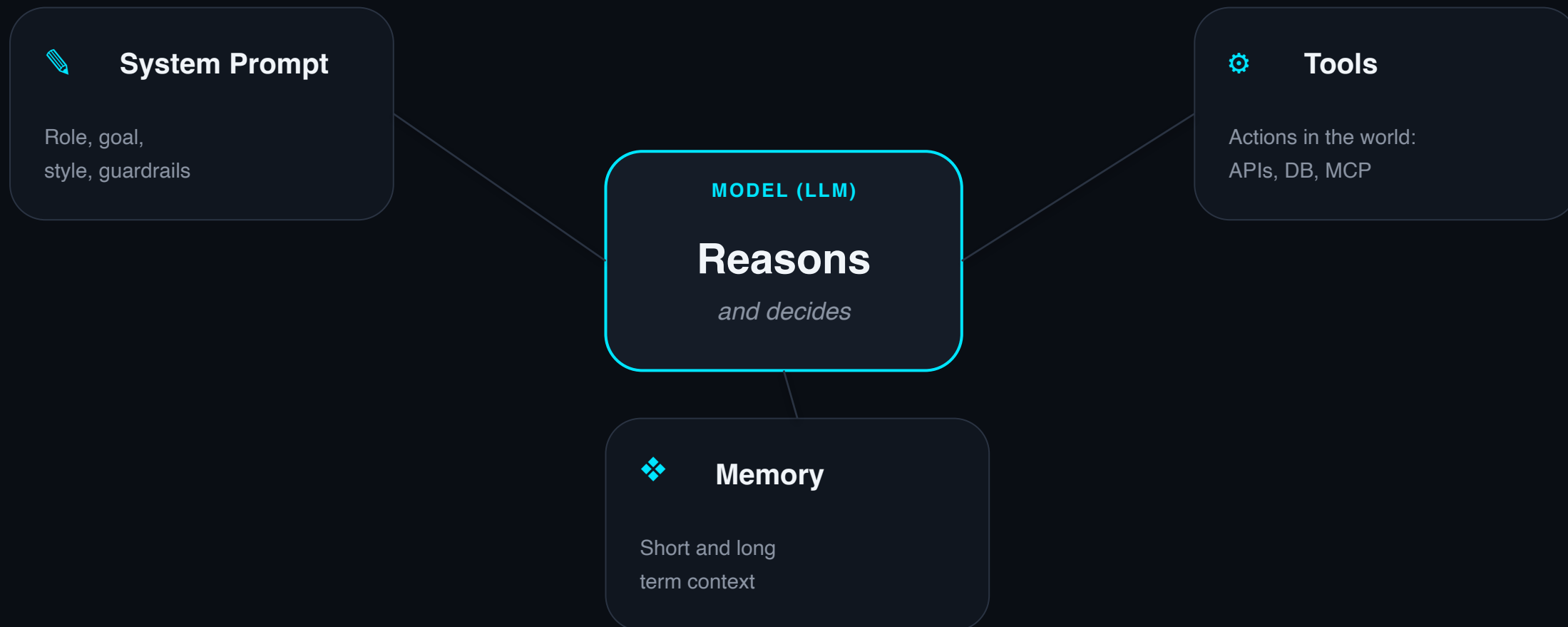
In-context learning + user feedback. Without feedback loops, the agent repeats its mistakes.



Resilience

Error handling, timeouts, attacks and unexpected conditions. Graceful degradation.

The 4 components of an agent



Choosing a model · 4 decision dimensions

The model defines how the agent interprets inputs, generates outputs and adapts. It shapes performance, latency and cost.

// 01

TASK COMPLEXITY

Open-ended and ambiguous

Frontier (GPT-5, Claude Opus 4.1) for complex, multi-step reasoning and creativity.
Small / distilled (Phi-4, ModernBERT) for repetitive, well-defined tasks.

// 02

MODALITY

Text · image · audio · video

Is the input only text, or also images, audio, documents, video?
Current frontier models (GPT-5, Claude 4.1) are multimodal by default.

// 03

OPENNESS & CUSTOM

Proprietary API vs open weights

Open (Llama, DeepSeek) for control, compliance and fine-tuning in sensitive domains.
Pretrained general for prototyping and generic cases; custom-trained where domain precision is critical.

// 04

COST & LATENCY

The operational decision

Large model = cloud + latency + \$\$\$. Small = local + fast + cheap.
Hybrid: large for complex, small for routine. Dynamic routing per query.

Frontier vs small · who fits which case

The most important dimension of the framework. The final answer is almost always “both”.

// FRONTIER · LARGE FOUNDATION

Open-ended · complex reasoning

GPT-5 · Claude Opus 4.1 · Gemini Ultra · DeepSeek V3

- + Generalization, nuance, creativity.
- + Sophisticated, multi-step tool use.
- + Handles ambiguity and long context.
- High cost per token, cloud required.
- Higher latency per call.

// USE CASES

Personal assistants · research agents · enterprise systems with unpredictable queries.

// SMALL · DISTILLED · EFFICIENT

Structured · fast · local

Phi-4 · ModernBERT · Gemma 2 · Llama 3.1 8B

- + Local hardware (consumer GPU).
- + Low latency and predictable cost.
- + Offline, privacy and compliance.
- Less flexible with novel inputs.
- Fine-tuning sometimes required.

// USE CASES

Guided customer support · information retrieval · data labeling · classification and extraction.

Open weight · MMLU vs required hardware

What I can self-host. Source: HELM (Stanford), MMLU as a general proxy.

Model	Vendor	MMLU	Params (B)	VRAM (GB)	Hardware
Llama 3.1 Instruct Turbo	Meta	56.1	8	20	RTX 3090
Gemma 2	Google	72.1	9	22.5	RTX 3090
NeMo	Mistral	65.3	12	24	RTX 3090
Phi-3	Microsoft	77.5	14.7	29.4	A100
Qwen 1.5	Alibaba	74.4	32	60.1	A100
Llama 3	Meta	79.3	70	160	4 × A100

▶ consumer GPU reaches up to ~12B parameters (RTX 3090 with 24 GB).

▶ 70B requires 4 × A100 · datacenter or cloud.

Flagship · performance vs relative price

Relative price per 1M tokens (baseline: Llama 3.1 Turbo 405B = 1x).

Model	Vendor	MMLU	Input (x)	Output (x)
DeepSeek-v3	DeepSeek	87.2	2.75	3.65
Claude 4 Opus Ext. Thinking	Anthropic	86.5	75	125
Gemini 2.5 Pro	Google	86.2	12.5	25
Llama 3.1 Instruct Turbo 405B	Meta	84.5	1	1
o4-mini	OpenAI	83.2	5.5	7.33
Nova Pro	Amazon	82.0	4	5.33
Mistral Large 2	Mistral	80.0	10	10
Grok 3	xAI	79.9	15	25

DEEPSEEK V3

best MMLU · ~3x the Llama baseline.

CLAUDE 4 OPUS

+0.7% MMLU vs DeepSeek · ~30x more expensive.

HYBRID ROUTING

go big only when it pays off.

System Prompt — 6 key principles

// 01

Clear Role Definition

“You are v0, Vercel's assistant.”
Anchors behavior.

// 02

Structured Organization

Headers, <thinking> tags,
Markdown lists.

// 03

Explicit Tool Integration

When to call each tool,
I/O schema, errors.

// 04

Chain of Thought

Think before acting.
Mandatory “thinking” phase.

// 05

Environment Awareness

OS, cwd, available shell.
Agent grounding.

// 06

Domain Constraints

“Use Tailwind”, “no package.json”.
Hard technical rules.

The system prompt is code: versioned, tested and measured against your evals.

Lessons from agents in production

Vercel v0

MDX COMPONENTS AS TOOLS

Instead of API calls, emits tags like `<CodeProject/>` or `<QuickEdit/>` to trigger UI changes. “No Apology” policy.

same.new

STRICT TOOL ETIQUETTE

Forbids mentioning tool names to the user. Requires explaining why a tool is being invoked before doing so.

Manus

EXPLICIT PLAN-ACT-OBSERVE LOOP

Mandatory cycle: gather information, process, reflect on the result before the next step.

Bolt.new

ENVIRONMENT CONSTRAINTS

Details what the WebContainer can and cannot do: “no Python”, available commands, explicit limits.

Each team crystallizes its agentic architecture in the system prompt.

Tools — the agent's hands

The functional building blocks. Modular design → update without rewriting the agent.



Local tools

Agent-internal logic.
Calculations, rules, local DB,
pure functions.



API-based tools

External services in real
time. CRM, payments, weather,
REST/GraphQL third parties.



MCP

Model Context Protocol:
standard schema to inject
context, memory and state.

BEST PRACTICES

Each tool is a risk surface: validate inputs, authorize permissions, audit calls and bound scope. Replaceable modules, not monoliths.

Defining a tool: LangChain + @tool

```
// tools/stock_price.py
```

```
1 from langchain_core.tools import tool
2 from langchain_openai import ChatOpenAI
3 from langchain_core.messages import HumanMessage
4 import requests
5
6
7 @tool
8 def get_stock_price(ticker: str) -> float:
9     """Get the stock price for a given ticker."""
10    url = f"https://api.example.com/stocks/{ticker}"
11    response = requests.get(url)
12    if response.status_code == 200:
13        return response.json()["price"]
14    raise ValueError(f"Failed to fetch {ticker}")
```

01 · DECORATOR

@tool

Turns a Python function into a tool the LLM can invoke.

02 · SCHEMA

Type hints

ticker: str → float defines the contract the model sees.

03 · INTENT

Docstring

The LLM reads it to decide WHEN to call the tool.

Tool-calling loop: perceive → reason → act

```
// agents/stock_agent.py
```

```
1 llm = ChatOpenAI(model_name="gpt-4o", temperature=0)
2 llm_with_tools = llm.bind_tools([get_stock_price])
3
4 messages = [HumanMessage("What is the stock price of Apple?")]
5
6 ai_msg = llm_with_tools.invoke(messages)
7 messages.append(ai_msg)
8
9 for tool_call in ai_msg.tool_calls:
10     tool_msg = get_stock_price.invoke(tool_call)
11     messages.append(tool_msg)
12
13 final_response = llm_with_tools.invoke(messages)
14 print(final_response.content)
```

01 · BIND

bind_tools([...])

Registers the tools. The model knows what it can call and with which schema.

02 · DECIDE

ai_msg.tool_calls

The model decides WHICH tool to invoke and with what arguments. It doesn't execute.

03 · LOOP

Perceive → reason → act

We append the observation and re-invoke until the final answer.

Memory — context and learning

SHORT-TERM

Immediate task context

- Rolling context window / scratchpad.
- Coherence during the session.
- Drops the old, keeps the relevant.
- Example: a chatbot that remembers your last questions in the conversation.

LONG-TERM

Persistent knowledge

- Databases, knowledge graphs, vector DBs.
- Preferences, history, learned facts.
- Enables personalization over time.
- Example: a health agent tracking vital signs for years.

Indexing

Organize for fast retrieval:
keywords, embeddings, metadata.

Relevance

Tell signal from noise.
Scoring + contextual filters.

Forgetting

Knowing what to forget is a feature.
TTL, retention, privacy.

How capabilities and agents coordinate

Orchestration turns isolated capabilities into end-to-end solutions: it composes, plans and supervises tool sequences toward a goal.

①

Single Agent

One agent with many tools.
Simple and predictable.



Router / Supervisor

An agent routes to
specialists.



Sequential Pipeline

Chained agents:
output → input.



Parallel Fan-out

Parallel branches
with final reduce.



Hierarchical

Manager → sub-agents.
Nestable.



Graph / Stateful

State machine with
loops and conditions.

Without orchestration, the best skills end up pulling in opposite directions.

The 4 trade-offs that define your agent

// PERFORMANCE

Speed $\rightleftharpoons$ Accuracy

Fast, approximate response, then a slower, more precise refinement (hybrid strategy).

// SCALABILITY

Concurrency $\rightleftharpoons$ Cost

On-demand dynamic GPU, autoscaling, off-peak scheduling.

// RELIABILITY

Consistency $\rightleftharpoons$ Complexity

Fault tolerance, redundancy, continuous monitoring and edge-case testing.

// COST

Capability $\rightleftharpoons$ Budget

Expensive models only for mission-critical work. Dynamic routing for the rest.

Single-Agent vs Multi-Agent

// SINGLE-AGENT

One agent, many tools

PROS

- Simple to design and operate.
- No coordination overhead.
- Predictable and easy to evaluate.

LIMITS

- Saturates with very diverse tasks.
- Monolithic design: hard to evolve.

// MULTI-AGENT

A team of specialized agents

PROS

- Specialization by domain.
- Real parallelism.
- Redundancy and resilience.

COSTS

- Communication = more tokens, more cost.
- Complex coordination, more operational debt.

Rule: start single-agent. Move to multi-agent only when the pain demands it.

Design patterns you'll want to know

// 01

ReAct

Reason + Act in a loop:
think → act → observe.

// 02

Plan-and-Execute

Plan first,
then execute step by step.

// 03

RAG

Retrieval-augmented
generation for grounding.

// 04

Reflection / Critic

The agent critiques its
own output before answering.

// 05

Multi-agent debate

Agents contrast
positions and decide.

// 06

Tool-first

Always try to use tools
before answering from memory.

// 07

Guardrails

Input/output classifiers.
Validate inputs and outputs.

// 08

Human-in-the-loop

Human approval at
critical steps.

OpenClaw · an OS for agents, not a wrapper

Treats the assistant as an infrastructure problem: sessions, memory, sandbox, routing... not just prompts.

// CHANNELS

Messaging adapters

WhatsApp · Telegram · Discord
Slack · iMessage · Signal

// CONTROL UI

Human interfaces

Web UI · CLI · macOS app
Mobile nodes (camera, voice)

// TRIGGERS

Proactive

Cron jobs · Webhooks
Voice wake word

// HUB

Gateway

WebSocket · 127.0.0.1:18789 · typed JSON Schema

- Single control point per host.
- Auth, pairing and access control.
- Resolves session (main / dm / group).
- Dispatches to Agent Runtime.
- Event-driven: agent · presence · tick.
- Idempotency keys for side effects.

// RUNTIME

Agent Runtime

Pi Agent Core · streaming

- 1 · Resolve session
- 2 · Assemble context
- 3 · Stream + tool calls
- 4 · Persist state

// STATE

Your disk, your data

~/openclaw/sessions/ · append-log
~/openclaw/memory/ · sqlite+vec
~/openclaw/credentials/ · 0600
workspaces/ · AGENTS.md · skills/

What patterns this agent OS combines

Six architectural decisions that define how OpenClaw orchestrates sessions, models and tools.

// 01

Hub-and-spoke

A single Gateway per host as control plane.
Event-driven WebSocket, no polling.
Typed protocol (TypeBox + JSON Schema).

// 02

Multi-agent routing

Each channel or group → isolated agent.
Per-session workspace, model and tone.
agents.mapping in config, not code.

// 03

Composable prompt

Per turn: AGENTS.md + SOUL.md + TOOLS.md.
Skills injected selectively (not all).
Semantic memory (SQLite + vec + BM25).

// 04

Agent-to-Agent

sessions_list · sessions_send · sessions_history.
sessions_spawn: create sessions to delegate.
Agents coordinate across channels and devices.

// 05

Per-session trust boundary

main = native host, full access.
DM / group = Docker sandbox by default.
Blast radius bounded to the container.

// 06

Proactive, not just reactive

Cron jobs: the agent speaks first.
Webhooks: Gmail, Sentry, IoT trigger actions.
Canvas / A2UI: the agent renders its own UI.

How to take an agent to production



Iterative Design

Don't chase perfection on the first build. Small prototypes, measure, refine.



Evaluation Strategy

Eval sets + functional tests. Boundary testing. Domain metrics. Golden datasets.



Real-World Testing

Controlled deploy with real users. Explicit feedback (NPS, thumbs) and implicit (logs).

PRODUCTION CHECKLIST

- ✓ Version prompts and tools
- ✓ Per-request budget + alerts
- ✓ Guardrails against prompt injection / PII
- ✓ Traces per step (tokens, cost, latency)
- ✓ Bounded fallbacks, timeouts and retries
- ✓ Feedback loop from day 1

Frameworks

FRAMEWORKS

- LangChain: designed to build sequential pipelines
- LangGraph: evolution of LangChain to support cyclic graphs systems
- Koog: Kotlin-based framework. Supports tools, complex graphs and more.
- AutoGen: Powerful multiagent orchestration; dynamic role assignment; flexible messaging-based interaction between agents
- A lot more.

Impact on Development

Developers, teams and the return of the architect.

How the developer's day-to-day changes

// YESTERDAY

- Writing all the code manually.
- Following approved patterns to the letter.
- Focus on syntax, boilerplate, repetition.
- Slow human reviews.
- Iteration paced by builds and releases.

// WITH AGENTS

- Co-design with agents: prompts as specification.
- Less boilerplate, more design decisions.
- New skills: evals, guardrails, AI observability.
- Tests and reviews as living guardrails.
- Much shorter feedback cycles.

The developer stops “writing code” and starts “directing systems that write code”.

Why the architect's role gains strength



More decisions, harder ones

Models, prompts, tools, memory, orchestration, costs: the design space explodes.



Non-trivial trade-offs

Quality vs cost vs latency vs security. Someone must reason about them explicitly.



New risks

Prompt injection, hallucinations, data leakage, non-deterministic behavior.



AI executes, humans decide

Agents accelerate the “how”. The architect protects the “what” and the “why”.

Key takeaways

// 01

Everything is a trade-off

Applies to agents the same way as to traditional systems. Make them explicit.

// 02

The -ilities rule

What decides architecture are not features, but constraints and quality attributes.

// 03

Agents are systems

Model + prompt + tools + memory. Apply engineering, not magic.

// 04

The architect protects the why

AI accelerates the how; humans decide what and why.

● THANK YOU

Questions?

Let's talk about architecture, agents and trade-offs.

Danilo Domínguez · Software Architect